

Hashing Based Table

By Nicholas and Eniola

Before I begin this essay, I'd like to admit that this entire project was done last minute. This is my own fault and I recognize that had I given the project more time, it would be a lot more polished. However, I am still proud of the work that both Eniola and I have done to finish this project.

Choosing the topic

A requirement for this project is to think of a simulated use case for the hash table. Eniola and I decided to follow along with the given example on Canvas (the phonebook idea) and make a contact list akin to the one you might see on your smart phone. A hash table would make a great use case out of this because everyone you put in is unique, and some people can get extremely large contact lists on their phones, so it's important to have quick access to the information where it's possible for a user to have hundreds of contacts to look through. A typical list and search algorithm can take a noticeable amount of time compared to a hashmap with an $O(1)$ time complexity in its search function given proper implementation.

Implementation

I began by creating a repository on GitHub for Eniola and I to access and collaborate on the code together. I then started by adding a file for the main function and a class for the data nodes, named "Contact". The Contact class is responsible for holding the data regarding the individual contacts on one's phone. I gave it the standard variables to hold a name, phone number, and email address along with the standard accessors and mutators.

```

#pragma once
#include <string>

class Contact{
private:
    unsigned long long phoneNum;
    std::string email;
    std::string name;

public:
    Contact();
    Contact(std::string newName, unsigned long long newPhone, std::string email);

    std::string getName();
    unsigned long long getPhoneNum();
    std::string getEmail();

    void setName(std::string newName);
    void setPhoneNum(unsigned long long newPhone);
    void setEmail(std::string email);

    std::string toString();
};

```

My only problem with the creation of this class was that during testing, I found that a standard integer did not give enough room to store a number with ten digits. While i may have been able to use a string in hindsight, I found that the most straightforward option was to increase the space given for the phone number. A long long variable is overkill for a phone number, however an unsigned long can only hold a maximum value of about 4.2 billion. A phone number can theoretically have 10 billion possible values (0 to 9,999,999,999 inclusive) so I had to use a long long to accommodate the high values.

While I was creating the Contact class, Eniola began work on the hashmap class to manage the nodes. I cannot speak to his experiences developing the hashmap class. However I can speak to the issue that arose last minute. While we were reviewing the sample code, we didn't see a hashing function that was used in the addRecord method. We both assumed that a hashing function wasn't needed, and just gave the unordered map object an integer to store the ID in. This worked, but only for a single value as we now realized that it had a default ID value that it was giving each data node meaning that any node after the first was overriding the existing data node. I quickly made a hash function to create an ID for the data nodes.

```

int HashTableApp::hash(Contact con)
{
    // Probably a bad hash function but this is what I could think of quickly
    std::string name = con.getName(); // get name
    int id = 0;

    for (int i = 0; i < name.length(); i++)
    {
        id += static_cast<int>(name[i]); // add the ASCII values of all the letters in the name
    }
    id = id * name.length(); // multiply the id by the length of the name
    name.length() % 2 == 0 ? id = id * -1 : id = id; // If name is of even length, make the ID negative

    return id;
}

```

This is not likely the best hash function. I haven't investigated it, but I'm confident that collisions are likely if given a large data set. It works by taking the ascii value of each character, adding them together, multiplying the result by the number of character in the name string, and inverting the number if there are an even number of characters in the name string. It could be better if it incorporated the other information of the contact like the phone number and/or the email address. However, the given method is functional for this project.

```

ID: -3600
Eniola:
5552223333
eni@captechu.edu

ID: -1556
Nick:
5557771111
nick@captechu.edu

ID: 2505
Mehri:
5551239876
mehri@captechu.edu

```

After testing the other functions, the implementation of the hash table is complete. Again, I recognize that leaving this project until the last minute was not the best decision, however I am still proud of the final product that we have created under the time crunch. My biggest insight into what I would do better for next time is just to research how to make a good hashing function, and more importantly, how to handle collisions because no hashing function is going to be 100 percent perfect at keeping all entries in a unique ID.